

Reviewer Pack — Minimal Local Coherence Index in Noncrossing Partitions and ...

Entry 696243c1f705 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-08 13:17:14 UTC

Minimal Local Coherence Index in Noncrossing Partitions and Circuit Topology Inequality

Entry ID: 696243c1f705 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-08 13:17:14 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Topology (Local Systems) **Field B** (complexity object): Boolean Circuits (Circuit Topology)

Statement:

For every Boolean circuit C with n inputs, the minimal local coherence index of its associated noncrossing partition is $O(\log(n)) * \text{depth}(C)$.

Rationale (proposer's reasoning):

Noncrossing partitions encode a natural ordering that can be used to measure the complexity of circuits. The local coherence index captures the complexity of this ordering and provides a potential link between algebraic topology and circuit theory.

Taxonomy category: AlgebraicTopology (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 450aeafdce4785c5

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a Boolean circuit C with n inputs, if the associated noncrossing partition's minimal local coherence index is within $O(\log(n)) * \text{depth}(C) \pm 3$ for all seeds and the support fraction is ≥ 0.8 .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - 'noncrossing partitions' AND 'circuit topology inequality' AND 'algebraic topology' - 'local coherence index' AND 'Boolean circuits' AND 'algebraic topologies' - 'depth of Boolean circuits' AND 'noncrossing partition complexity'

Top relevant hits considered: - [http://arxiv.org/abs/2212.13799v6] Noncrossing partitions of a marked surface - [http://arxiv.org/abs/1604.06009v2] Oriented Flip Graphs and Noncrossing Tree Partitions - [http://arxiv.org/abs/1904.05573v3] k -Indivisible Noncrossing Partitions - [http://arxiv.org/abs/1401.3218v1] Ground State Quantum Coherences: from Quantum Beats to Strong Control - [http://arxiv.org/abs/2507.16123v3] Einstein's Electron and Local Unitary Branching: Boundaries of Islands of Coherence and Quantum Nonlocality - [http://arxiv.org/abs/1702.06061v4] Generalized Coherence Concurrence and Path distinguishability - [http://arxiv.org/abs/1407.6169v3] Multiplicative Complexity of Vector Valued Boolean Functions - [http://arxiv.org/abs/1606.05050v1] Proof Complexity Lower Bounds from Algebraic Circuit Complexity

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_circuit(n):
        if n == 0:
            return []
        elif n == 1:
            return [random.choice([0, 1])]
        else:
            left_size = random.randint(1, n-1)
            right_size = n - left_size
            left = generate_circuit(left_size)
            right = generate_circuit(right_size)
            return [left, right]

    def depth(circuit):
        if isinstance(circuit, list):
            return 1 + max(depth(subcircuit) for subcircuit in circuit)
        else:
            return 0

    def noncrossing_partition(circuit):
```

```

    if isinstance(circuit, list):
        left = noncrossing_partition(circuit[0])
        right = noncrossing_partition(circuit[1])
        return [left + right]
    else:
        return []

def local_coherence_index(partition):
    if not partition:
        return 0
    return sum(len(subpartition) for subpartition in partition)

n_values = [5, 10, 15, 20, 30, 40]
results = []

for n in n_values:
    circuit = generate_circuit(n)
    depth_val = depth(circuit)
    partition = noncrossing_partition(circuit)
    msl = local_coherence_index(partition)

    if msl == 0 or depth_val == 0:
        continue

    results.append({
        "metric_name": "msl_over_depth",
        "metric_value": msl / depth_val,
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": False if msl > 3 * math.log(n) * depth_val else True,
        "counterexample": ""
    })

mean_msl = sum(result["metric_value"] for result in results)
std_dev = math.sqrt(sum((result["metric_value"] - mean_msl) ** 2 for result in results) / len(results))
support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

return {
    "seed": seed,
    "mean_msl_over_depth": mean_msl,
    "std_dev_msl_over_depth": std_dev,
    "support_fraction": support_fraction
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_msl = sum(result["mean_msl_over_depth"] for result in results) / len(results)
    std_dev = sum(result["std_dev_msl_over_depth"] for result in results) / len(results)
    support_fraction = sum(result["support_fraction"] for result in results) / len(results)

```

```

if all(result["conjecture_holds"] for result in results):
    print(f"RESULT: SUPPORTED mean={mean_msl} std={std_dev} support_fraction={support_fraction}")
elif any(not result["conjecture_holds"] for result in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"msl > 3 * log(n) * depth(C)\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_899858ec.py", line 89, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_899858ec.py", line 58, in run_trial
    partition = noncrossing_partition(circuit)
               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_899858ec.py", line 41, in noncrossing_partition
    left = noncrossing_partition(circuit[0])
          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_899858ec.py", line 41, in noncrossing_partition
    left = noncrossing_partition(circuit[0])
          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_899858ec.py", line 41, in noncrossing_partition
    left = noncrossing_partition(circuit[0])
          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

```

[Previous line repeated 1 more time]

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_899858ec.py", line 42, in noncrossing_partition
    right = noncrossing_partition(circuit[1])
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

```

IndexError: list index out of range

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means it did not complete its execution to verify the conjecture. | next: Investigate and fix the crash in the test code to allow for a proper verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13443
2	preregistration	ollama_remote	glm4:latest	0	0	9685
3	novelty	ollama_remote	glm4:latest	0	0	8752
4	novelty	ollama_remote	glm4:latest	0	0	18046
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	64147
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10899
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9226
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10764
9	judge	ollama_remote	glm4:latest	0	0	22988

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 167950 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/696243c1f705.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/696243c1f705.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/696243c1f705.tar.gz` (if generated)